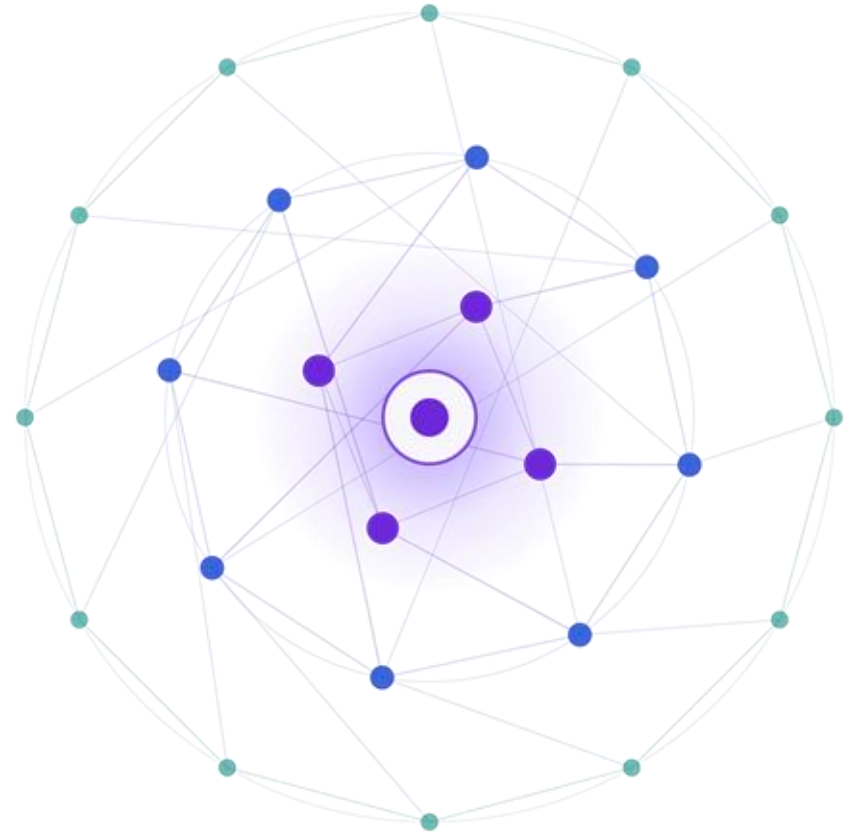


SAMSUNG PRISM

Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - 1
- Team Name - Genovate
- College Name - SRMIST
- Member Name & Email 1- Ipshita Sharma (is7675@srmist.edu.in)
- Member Name & Email 2- Divita Kapoor (dk3413@srmist.edu.in)
- Submission Github link -



Theme

Agentic Code Intelligence

Retrieval of the right code, not just similar code

The problem

AI agents fix bugs and add features only if they first reach the right files and code blocks; that problem grows as codebases expand.

What is code retrieval?

- Input: a library of code snippets + a natural-language query
- Output: snippets ranked by relevance to the query
- Example: *"How is the input preprocessed before going to the main function?"*

Why it matters for AI agents

- Retrieval is the first step of every agent workflow. If it misses, everything downstream fails.

Our goal

A fast, CPU-first, multi-stage retrieval system that ranks code by developer intent, across thousands of snippets and across code versions.

Existing Solutions & Gaps

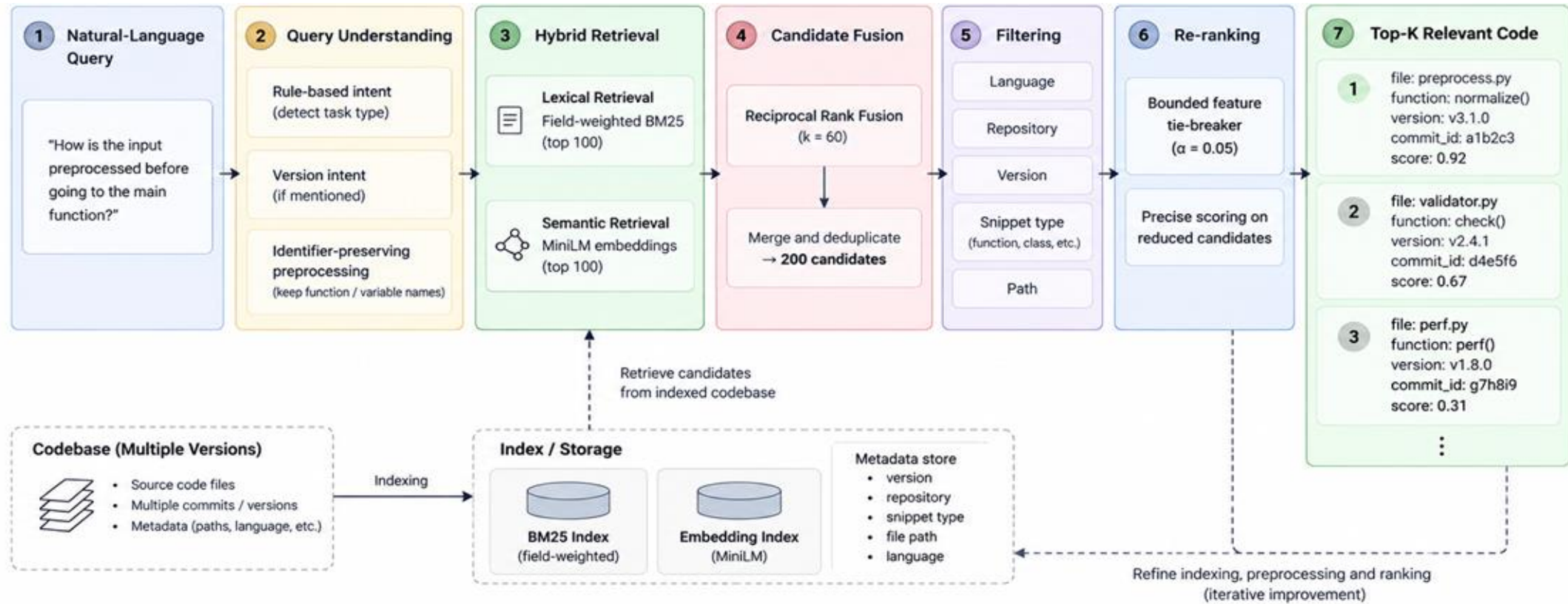
Approach	Strength	Limitation
Keyword / lexical search	Fast; exact identifier matching	Query words rarely match code tokens (preprocess vs normalise, trim)
Dense / embedding retrieval	Captures semantic meaning; runs on CPU with small models	Single pass; near-identical snippets and versions are hard to separate
LLM-based ranking	Strong relevance judgement	Snippets exceed the context window; too slow for a retrieval step

The gap

- No single method gives accuracy, speed and version support together.
- Snippets are numerous and long, and versions of the same snippet look almost identical.

We combine fast signals in stages:- narrow the candidates cheaply, then rank precisely.

Our Solutions & Architecture Diagram



What Makes Our Retrieval Intelligent

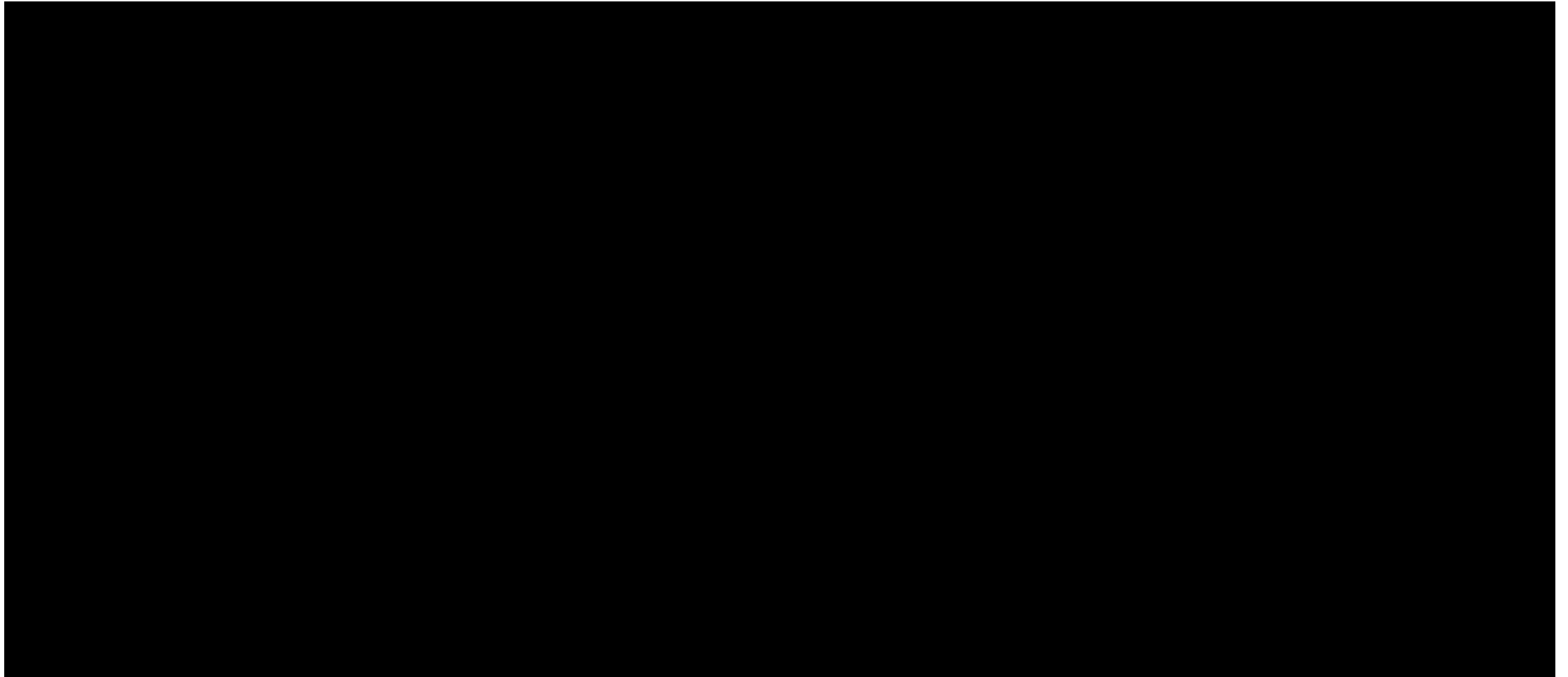
Multi-pass retrieval: Fast lexical + semantic recall first, precise scoring later.

Identifier-aware: Preserves distinctive identifiers like **normalize_input** during preprocessing.

Version-aware: Query intent such as **latest** or **v2** influences candidate filtering.

Explainable: Every result exposes lexical, semantic, fusion and re-ranking scores.

Demo & Product Walkthrough



Natural-language code retrieval
latency

Version & language filtering

Ranked results + retrieval

Tools and tech stack used

Category	Technologies
Retrieval	Field-weighted BM25 (custom, dependency-free) · sentence-transformers/all-MiniLM-L6-v2 · RRF fusion · NumPy dot-product search over normalized vectors
NLP / Query Processing	Rule-based intent classifier · version-intent parser · identifier-preserving tokenizer (camelCase / snake_case splitting) · technical alias expansion
Code Intelligence	Snippet model (function, class, docstring, imports, path, version, commit ID) · Python AST chunker · declaration-aware patterns for other languages · deterministic snippet IDs
Evaluation	MTEB 2.21.10 · CoIR AppsRetrieval (test split) · NDCG@10 · MRR · Recall@5 / @10 · pytest (53 tests)
Infrastructure	Python 3.11+ · NumPy · Pydantic · sentence-transformers 5.5 · disk-persisted index and embedding cache with corpus-hash invalidation

CPU-first by design

- No GPU, API key, or external LLM call.
- Batched, cached embeddings on CPU.
- Filtering and fusion happen before reranking.
- FAISS / HNSW can replace the in-memory search later, behind the same interface.

Impact & Use case

IMPACT

Retrieval quality sets the ceiling for every AI agent step that follows

locate → **understand** → **fix** → **extend**

- **Cleaner context:** the agent only sees the right snippets, not the whole repository
- **Fewer wrong-file edits:** edits start from the correct code
- **Faster fixes:** less time spent searching, more time spent solving
- **Scales with the codebase:** retrieval stays fast as snippets grow in number and length

Better retrieval gives downstream agents the right starting point, and everything after it improves.

USE CASES:

Use case	How retrieval helps
Debugging	Maps an error description to the responsible code
Codebase exploration	Turns a question into the right file and block
Unfamiliar repositories	Finds entry points without reading the whole repo
Feature development	Locates where new logic should plug in
Legacy code understanding	Surfaces intent in code that was never documented
Version-aware development	Retrieves code as it existed at any version
AI coding agents	Supplies precise context instead of the entire repository

Innovation highlights, results and limitations

A. INNOVATION

- **Field-weighted BM25:** function name, docstring, imports and path scored separately
- **Hybrid + RRF fusion:** BM25 and MiniLM candidates merged by ID
- **Query-conditioned version intent:** "v2" or "latest" in the query sets the filter
- **Bounded, explainable reranker:** capped tie-break ($\alpha = 0.05$) with a full score trace
- **Filter before rerank:** heavy scoring only on the reduced candidate set

B. RESULTS

Metric	CoIR AppsRetrieval (official, 3,765 queries)	Local sample (36 snippets, 42 queries)
NDCG@10	0.0673	0.9326
MRR	0.0552 (MRR@10)	0.9722
Recall@10	0.1068	0.9821
Latency	~0.59 s/query (8,765 docs)	14.3 ms mean

Ablation (6-query sample): BM25 only 0.779 → hybrid 0.770 → + reranking **0.792** NDCG@10

C. LIMITATIONS

- Official accuracy is low so far; improving it is the priority
- The reranker is a rule-based tie-breaker, not a learned model
- Full-corpus search runs on in-memory scoring, about 0.6 s/query
- Cross-file dependencies and evolutionary retrieval are not yet built

What's next

CURRENT

- Hybrid BM25 + MiniLM retrieval, RRF fusion, version-aware filtering, bounded reranker, MTEB-validated end to end.



NEXT

- Raise official accuracy: fine-tune the encoder on AppsRetrieval positives and hard negatives, tune fusion and BM25 weights on held-out qrels, and add a small cross-encoder over the top 50 candidates.



FUTURE

- Version-diff features for evolutionary retrieval over commit_id, tree-sitter symbol and reference metadata, and FAISS / HNSW for large-scale search.

Brownie points slide(differentiation)

Traditional code retrieval	Our approach
Keyword matching	Intent-aware retrieval
Single pass	Multi-stage retrieval
Semantic-only	Hybrid signals
Static index	Version-aware retrieval
One strategy for every query	Query-adaptive retrieval
Similarity	Relevance

- "Query-adaptive retrieval" becomes **"Query-conditioned version filtering and intent signals"**.
- **P0, Retrieval Accuracy:** hybrid, multi-stage, explainable ranking, evaluated with MTEB (official NDCG@10 = 0.0673; improving it is the next step).
- **P1, Retrieval Across Versions:** version=v1 / v2 / all / latest and query-detected version intent, with the commit ID preserved.
- **Bonus, Evolutionary Retrieval:** the version and commit metadata are in place, and version-diff ranking is the planned next layer.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y)
- README with reproducible setup instructions (Y)
- Demo video, max 5 minutes (Drive link)
- Presentation file (PDF) (Y)

Thank you